

Getting Started with the Android NDK



This article focuses on development for the Android platform using the Android Native Development Kit (NDK). Android applications are run on the Dalvik Virtual Machine, which means writing code in Java. The NDK lets developers implement parts of their applications in native code, like C and C++, which in turn, allows code to be executed faster.

Before we get started, you need to be comfortable with C, C++, Java and Android development. You need a working Android development environment and a fairly basic understanding of how Makefiles work.

When should you use the NDK?

An interesting question is, when do you actually use the Android NDK rather than the standard set of tools provided by the Android SDK? Well, you generally use the NDK when the problem you are trying to solve is CPU-intensive and the application would benefit from running natively—and also when you want to port code from another platform to Android. Applications that use libraries written in C and C++, like OpenGL, can be ported to Android relatively quickly and easily, using the NDK. Note that code performance will not increase just by writing native code. There is some overhead generated by Java and native code calls. Hence, you should only use the NDK when you are sure that the performance improvement is worth the overhead.

Getting started by downloading tools

The Android NDK, unlike most other Android tools, cannot be installed by using the Android SDK manager. Instead, you must download and install the package for the Android NDK separately. Get the latest version for your OS from <http://developer.android.com/tools/sdk/ndk/index.html>.

Installing the NDK

After downloading the package, you need to unzip it in an appropriate directory. You have to add the path to the unzipped NDK package to your system path (or work relative to the extracted NDK path). On most Linux systems, you can add the directory to your system path with the following command:

```
export PATH=$PATH:/path/to/extracted/NDK/package
```

Creating projects

You can create a regular Android project using your development environment. Once created, in the project folder, add another folder called *jni*, in which you place all your native code. JNI stands for the Java Native Interface, and the Android NDK is heavily based on concepts from the JNI; the NDK is fundamentally a reduced set of C headers from the JNI.

Getting your NDK app working

After set-up, it's time to start writing some native code. Let us look at a segment of code that is supposed to be processor-intensive—the generation of Fibonacci numbers using recursion. Let us write the Java implementation first, and the native implementation later. For practical purposes, let us assume that the project I created in the following example is *com.ndk*.